Online Shopping Purchase Prediction
Using Machine Learning

Problem Statement:

Online shopping websites receive visits from many customers, but every visitor does not complete a purchase. Some customers browse products, compare prices, view discounts, or add items to the cart and then leave the website without buying. Understanding these customer activities can help an online store improve its shopping experience and plan suitable offers.

Manual analysis of customer behaviour can be time-consuming when the number of visitors is large. Therefore, this mini project develops a simple machine learning system that predicts whether a customer is likely to purchase based on selected behaviour details such as age, time spent on the website, pages visited, products viewed, previous purchases, cart activity, and discount. The prediction is displayed through a web interface built using Flask, HTML, and CSS.

Objectives:

1. To prepare a labelled dataset containing customer behaviour details and purchase outcomes.
2. To train a Random Forest Classifier using the prepared dataset.
3. To save the trained model and reuse it for predictions.
4. To build a Flask backend and a simple web form for entering customer details.
5. To display a clear result indicating whether the customer is predicted to purchase.
6. To demonstrate the connection between a machine learning model, backend, and frontend.

Software / Hardware Required:

- Operating System: Windows 10/11 (also compatible with Linux/macOS)
- IDE: Visual Studio Code
- Programming Language: Python 3.x
- Machine Learning: scikit-learn (RandomForestClassifier)
- Data handling: pandas, NumPy
- Backend Framework: Flask
- Model saving: Joblib
- Frontend: HTML5 and CSS3
- Dataset: dataset.csv (small sample dataset prepared for this mini project)
- Hardware: Laptop/desktop with 4 GB RAM or above and a web browser

Project Directory Structure

The project folder, named “OnlineShoppingPrediction”, contains the training program, Flask application, CSV dataset, saved machine learning model, and HTML templates. The structure below matches the simple implementation used for this mini project.

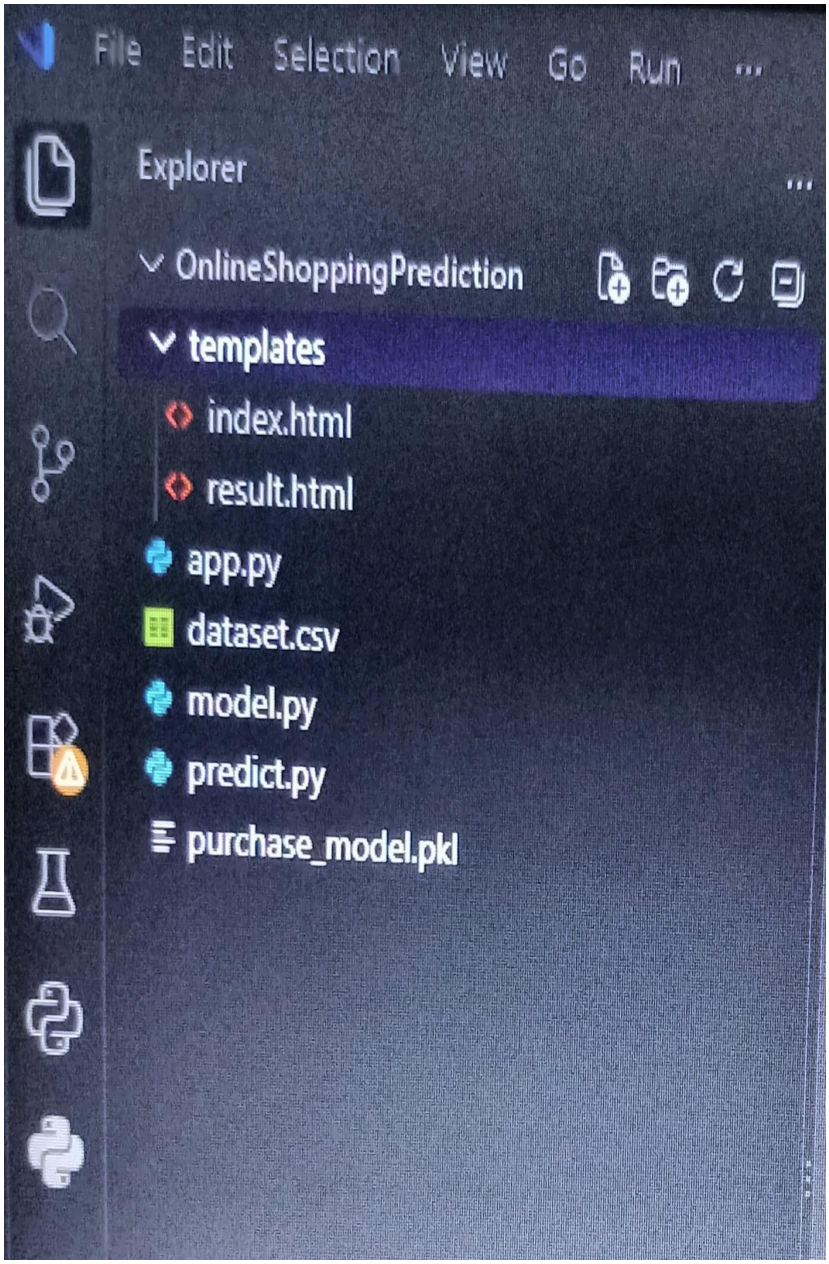


Fig 1: Project directory structure of OnlineShoppingPrediction in Visual Studio Code

Main Files and Their Purpose:

- app.py – starts the Flask web application, reads the form values, calls the model, and displays the prediction.
- model.py – loads dataset.csv, selects input columns, trains the Random Forest model, and saves purchase_model.pkl.
- predict.py – loads the saved model and tests a sample customer record from the terminal.
- dataset.csv – stores sample customer behaviour features and the Purchased label (1 = purchase, 0 = no purchase).
- purchase_model.pkl – trained model file created by model.py.
- templates/index.html – frontend page containing the customer input form.
- templates/result.html – displays the prediction and the Try Again link.

Procedure

The following procedure describes how the Online Shopping Purchase Prediction application is trained, launched, and tested.

- Create the project folder OnlineShoppingPrediction in Visual Studio Code.
- Create dataset.csv with the customer behaviour columns Age, TimeSpent, PagesVisited, ProductsViewed, PreviousPurchases, CartAdded, Discount, and Purchased.
- Install the required Python packages: pandas, NumPy, scikit-learn, Flask, and Joblib.
- Run model.py. The program reads the CSV dataset, separates input features from the Purchased label, trains the Random Forest Classifier, and saves purchase_model.pkl.
- Run predict.py to test a sample customer record and confirm that the saved model can make a prediction.
- Run app.py from the VS Code terminal. Flask starts a local development server at http://127.0.0.1:5000.
- Open the local address in a web browser. The home page displays the customer activity input form.
- Enter values for age, time spent, pages visited, products viewed, previous purchases, cart added, and discount.
- Click Predict Purchase. The form sends the entered values to the Flask /predict route.
- The backend passes the values to the saved Random Forest model and displays either “Customer is likely to PURCHASE” or “Customer is NOT likely to PURCHASE”.

Implementation Code

The following code shows the main model-training and prediction modules used in the project. The feature order in the input data must remain the same during both training and prediction.

1. Model Training (model.py)

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
import joblib
```

```
# Read the prepared customer dataset
data = pd.read_csv("dataset.csv")
```

```
features = ["Age", "TimeSpent", "PagesVisited", "ProductsViewed",
            "PreviousPurchases", "CartAdded", "Discount"]
X = data[features]
y = data["Purchased"]  # 1 = purchase, 0 = no purchase
```

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X, y)
joblib.dump(model, "purchase_model.pkl")
print("Model trained and saved successfully")
```

2. Standalone Prediction Test (predict.py)

```
import joblib
```

```
model = joblib.load("purchase_model.pkl")
customer = [[22, 18, 9, 6, 3, 1, 15]]
prediction = model.predict(customer)[0]
```

```
if prediction == 1:
    print("YES - likely to purchase")
else:
    print("NO - not likely to purchase")
```

Explanation:

RandomForestClassifier combines predictions from multiple decision trees. The model learns patterns from the example records in dataset.csv. Joblib saves the trained model to a file so that app.py and predict.py can load it without training again each time.

The example customer record is [22, 18, 9, 6, 3, 1, 15]: age 22, 18 minutes on the website, 9 pages visited, 6 products viewed, 3 previous purchases, cart added = 1, and a 15% discount. The displayed result depends on the trained model and dataset.

3. Flask Backend (app.py)

```
from flask import Flask, render_template, request
import joblib
```

```
app = Flask(__name__)
model = joblib.load("purchase_model.pkl")
```

```
@app.route("/")
def home():
    return render_template("index.html")
```

```
@app.route("/predict", methods=["POST"])
def predict():
    values = [[
        float(request.form["age"]),
        float(request.form["time_spent"]),
        float(request.form["pages_visited"]),
        float(request.form["products_viewed"]),
        float(request.form["previous_purchases"]),
        float(request.form["cart_added"]),
        float(request.form["discount"])
    ]]
    prediction = model.predict(values)[0]
    result = ("Customer is likely to PURCHASE ✅" if prediction == 1
              else "Customer is NOT likely to PURCHASE ❌")
    return render_template("result.html", result=result)
```

```
if __name__ == "__main__":
    app.run(debug=True)
```

4. Frontend Form (templates/index.html)

```
<form action="/predict" method="post">
  <input name="age" type="number" placeholder="Age" required>
  <input name="time_spent" type="number" placeholder="Time spent" required>
  <input name="pages_visited" type="number" placeholder="Pages visited" required>
  <input name="products_viewed" type="number" placeholder="Products viewed" required>
  <input name="previous_purchases" type="number" placeholder="Previous purchases" required>
  <input name="cart_added" type="number" min="0" max="1" placeholder="Cart added (0/1)" required>
  <input name="discount" type="number" placeholder="Discount (%)" required>
  <button type="submit">Predict Purchase</button>
</form>
```

5. Result Page (templates/result.html)

```
<h2>{{ result }}</h2>
<a href="/">Try Again</a>
```

The HTML form sends the entered values to the /predict route using the POST method. Flask obtains the values, converts them to numbers, and calls model.predict(). The result template then displays the message returned by the backend.

Output (Step-by-Step Screenshot)

Step 1 — Project Setup and Dataset:

The project files are created in Visual Studio Code. The CSV dataset contains one row for each sample customer and a Purchased column used as the target label. A value of 1 means the sample customer purchased; 0 means the sample customer did not purchase.

Age	TimeSpent	PagesVisited	ProductsViewed	PreviousPurchases	CartAdded	Discount	Purchased
22	18	9	6	3	1	15	1
35	4	2	1	0	0	0	0
28	12	7	4	2	1	10	1
42	3	2	1	1	0	5	0

Table 1: Example customer records (illustrative subset of dataset.csv)

Step 2 — Train and Save the Model:

Run the following command in the VS Code terminal:

```
python model.py
```

Expected terminal message: Model trained and saved successfully

After successful execution, purchase_model.pkl is created in the project folder. This file contains the fitted Random Forest model.

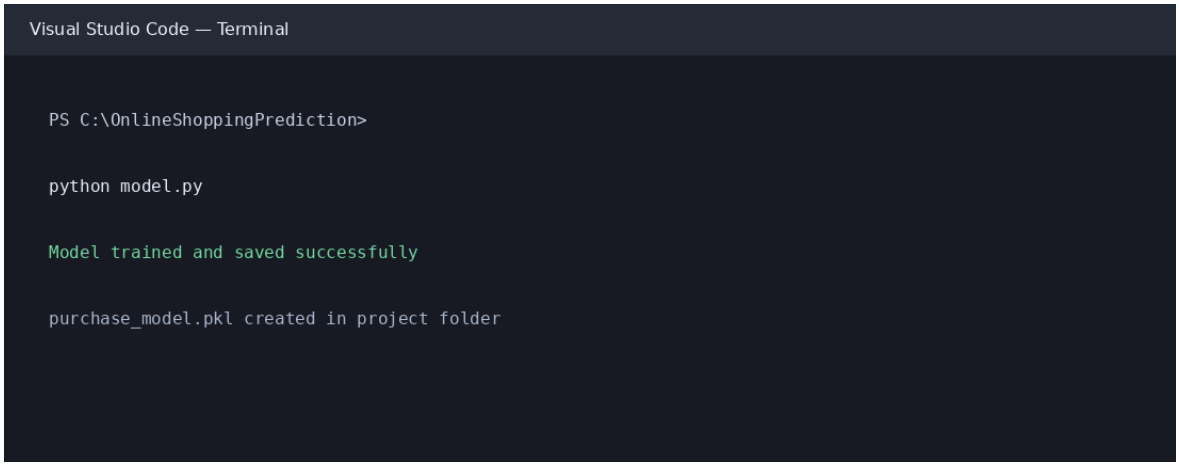


Fig 3: Example terminal output after model training completes

Step 3 — Launch the Flask Application:

Open the terminal in the OnlineShoppingPrediction folder and run app.py. The application loads the saved model and starts the local web server.

```
python app.py
 * Running on http://127.0.0.1:5000
```

Open the address in Chrome or another browser on the same computer. The Flask development server is local; the project does not need to be published online for this demonstration.

Step 4 — Customer Details Input Form:

The home page provides fields for the customer activity values. For the sample input shown below, age is 22, time spent is 18 minutes, pages visited is 9, products viewed is 6, previous purchases is 3, cart added is 1, and discount is 15%.

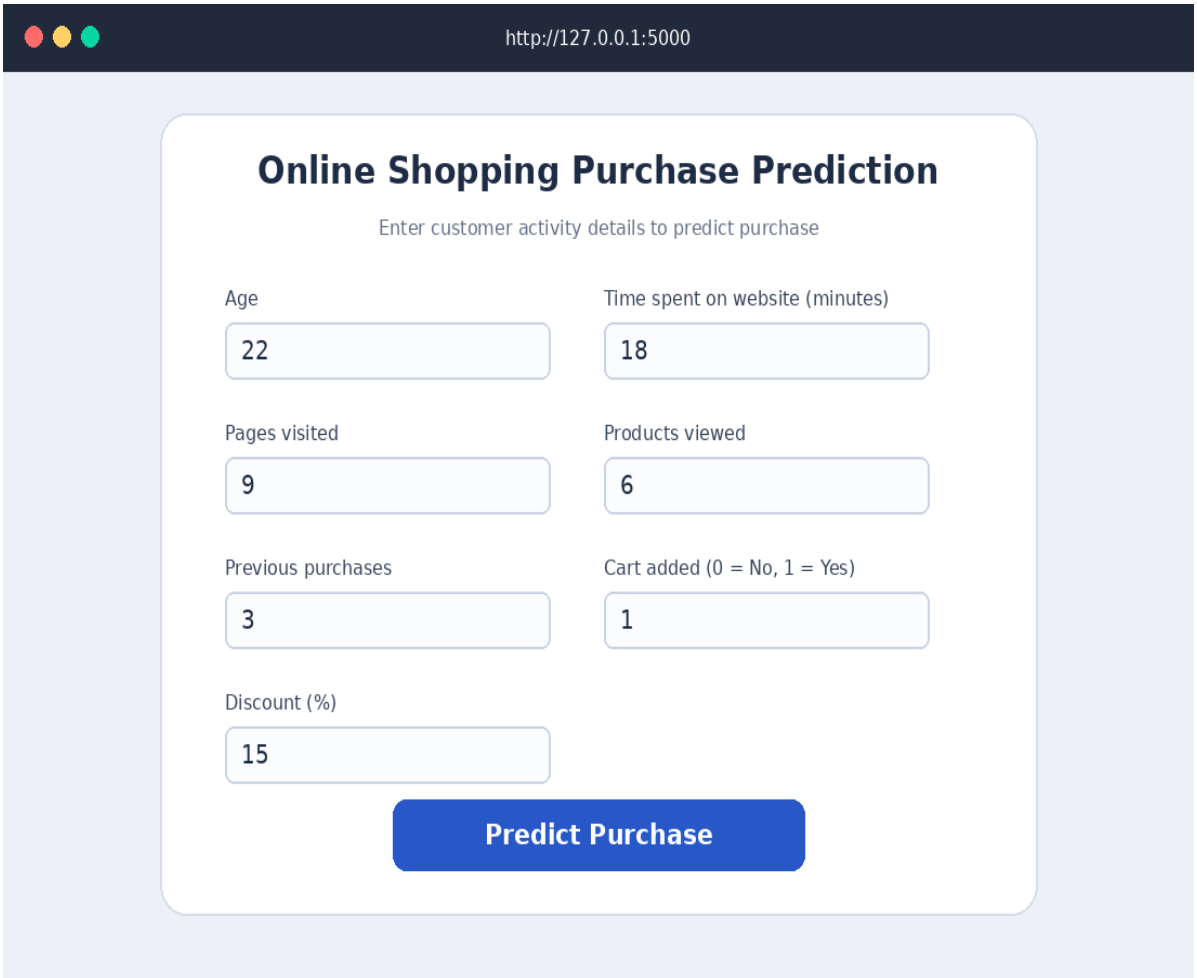


Fig 4: Customer details form shown in the browser (illustrative view of the implemented fields)

Step 5 — Passing Input to the Machine Learning Model:

When the user submits the form, the /predict route reads all seven fields and converts the values to numeric form. The values are arranged in the same feature order used while training the Random Forest model.

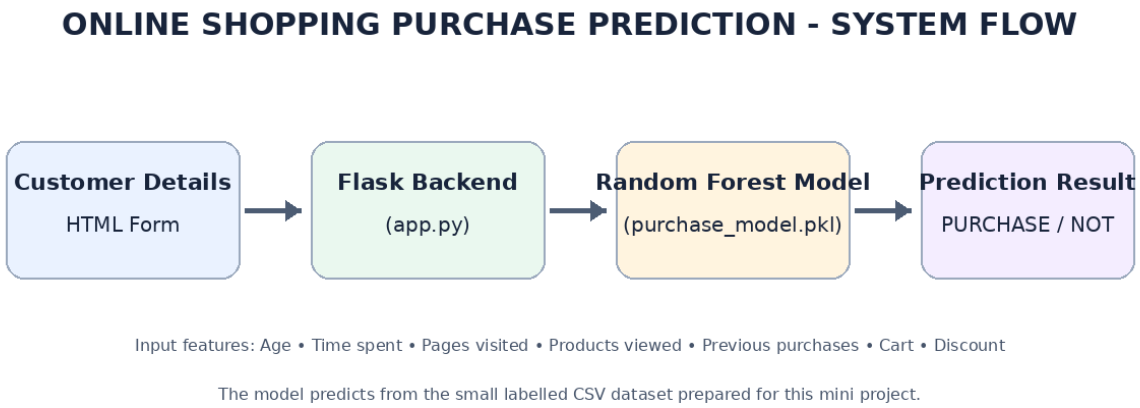


Fig 5: Flow of customer values from the frontend to the trained model and result page

Sample Input:

Age	22
Time spent	18 minutes
Pages visited	9
Products viewed	6
Previous purchases	3
Cart added	1 (Yes)
Discount	15%

The model returns a class label: 1 for likely to purchase or 0 for not likely to purchase. The web page converts that label into an easy-to-read message. This is a prediction, not a guarantee that a real customer will buy the product.

Step 6 — Positive Prediction Output:

For the example customer input, the model may return class 1 based on the patterns it learned from the sample dataset. The result page then displays “Customer is likely to PURCHASE” and provides a Try Again link to return to the input form.

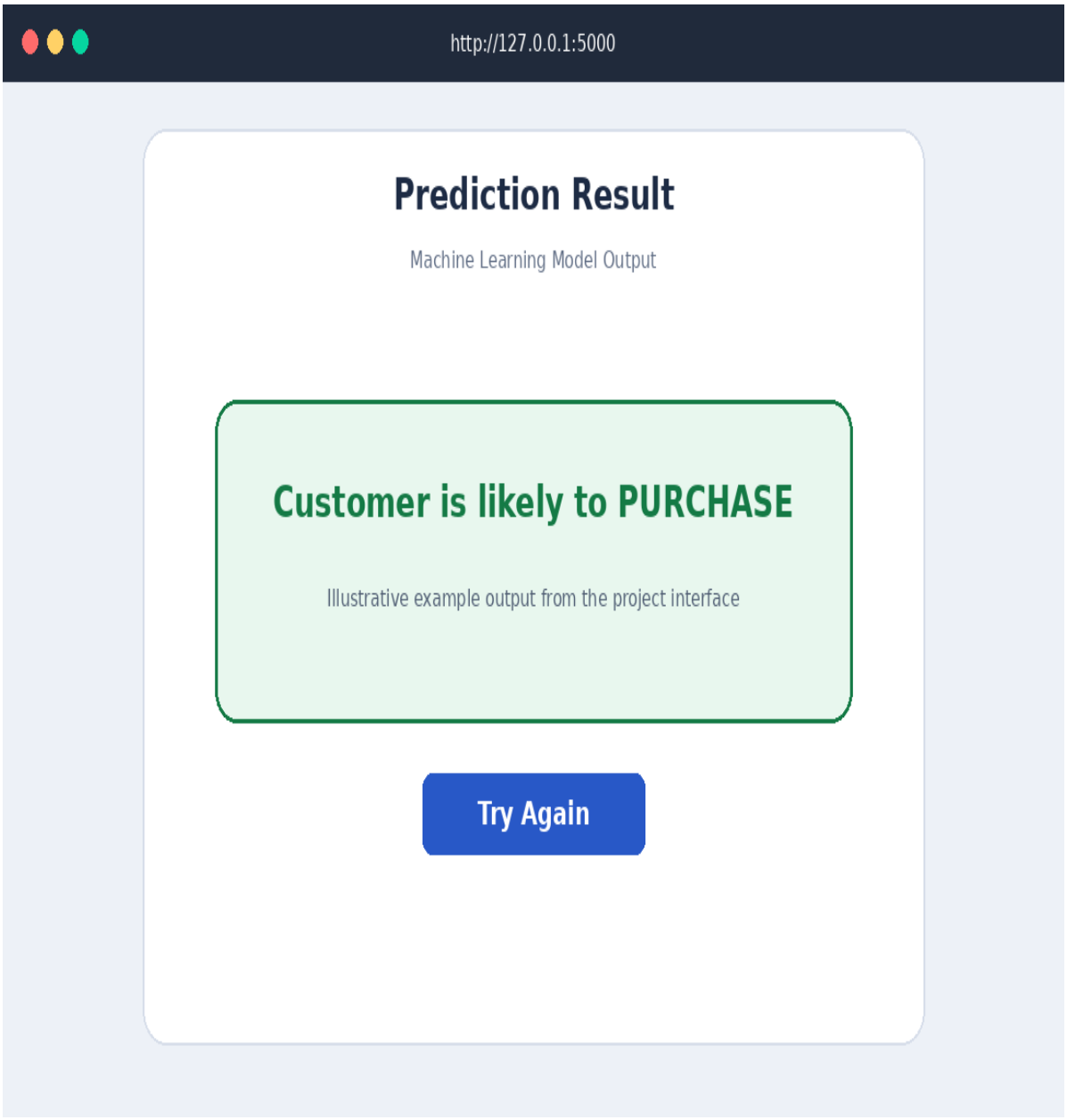


Fig 6: Example result page when the model predicts that the customer is likely to purchase

How to Read the Result:

- The green message represents a positive purchase prediction (class 1).
- Try Again returns the user to the input form for another prediction.
- The displayed class is generated by the saved Random Forest model using the submitted values.

This figure is an illustrative layout of the result screen, not a claim that the model has verified the future behaviour of a real customer.

Step 7 — Negative Prediction Output:

When the model returns class 0 for a set of customer details, the result page displays “Customer is NOT likely to PURCHASE”. The user can click Try Again and submit a different set of values.

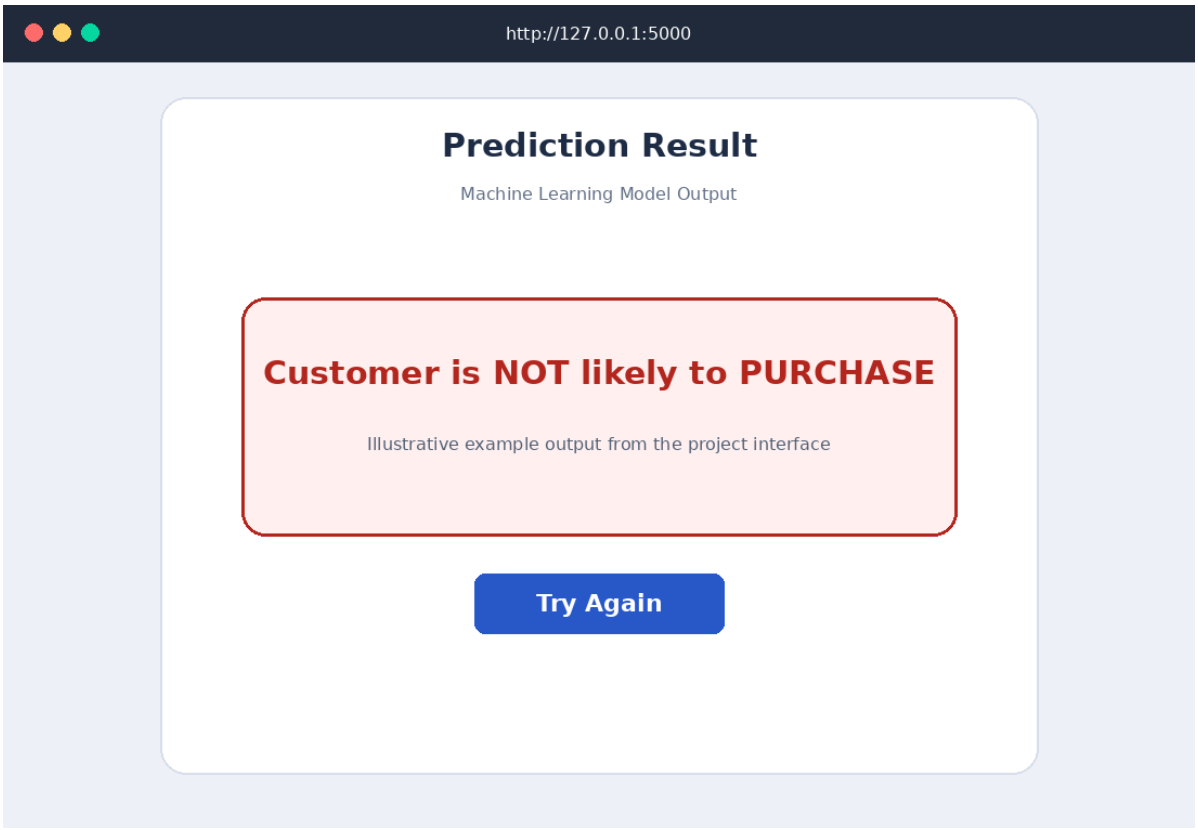


Fig 7: Example result page when the model predicts that the customer is not likely to purchase

Standalone Prediction Test:

The saved model can also be tested directly from the terminal by running predict.py. For the sample customer record used in the program, the terminal displays one of the following messages depending on the model prediction:

```
YES - likely to purchase
NO - not likely to purchase
```

Only the message produced by the actual run should be reported as the observed terminal output. Results can change if the dataset, training records, or model parameters are changed.

Result:

A basic web-based Online Shopping Purchase Prediction system was developed using Python, a Random Forest Classifier, Flask, HTML, CSS, a CSV dataset, and Joblib. The application accepts customer behaviour details through a web form, passes the values to the trained model, and displays whether the customer is predicted to purchase or not purchase. The trained model is saved as purchase_model.pkl and reused by the Flask application.

The project demonstrates how a machine learning model can be integrated with a frontend and backend in a simple web application. The user can enter customer details in the browser and receive a prediction without running the prediction code manually.